

# Teoría de Algoritmos

Problema de la Mochila 0/1 y Problema del Agente Viajero

Hernández Cuella Carlos Perusi

De la Cruz Flores José Rodolfo

Mendoza Roque Marcela

22 de mayo de 2026

## Introducción

La optimización combinatoria estudia problemas en los cuales se busca la mejor solución entre un conjunto finito, pero muy grande, de posibilidades. En este documento se analizan dos problemas clásicos:

- El Problema del Agente Viajero (TSP).
- El Problema de la Mochila 0/1.

Ambos pertenecen a la clase de problemas NP-difíciles, por lo que su resolución exacta resulta costosa para instancias grandes. Por ello, se emplean tanto algoritmos exactos como heurísticos y metaheurísticos.

## Problema del Agente Viajero (TSP)

### *Travelling Salesman Problem (TSP)*

El TSP consiste en encontrar la ruta más corta que recorra un conjunto de ciudades exactamente una vez y regrese al punto de origen.

## Planteamiento Matemático

Dado un conjunto de  $n$  ciudades y una matriz de costos  $M \in \mathbb{R}^{n \times n}$ :

- Conjunto de ciudades:

$$G = \{0, 1, \dots, n-1\}$$

- Ruta:

$$X = (x_0, x_1, \dots, x_{n-1})$$

- Función de costo:

$$C(X) = \sum_{i=0}^{n-2} M[x_i][x_{i+1}] + M[x_{n-1}][x_0]$$

- Generación de instancias:

$$M[i][j] = \begin{cases} 0 & \text{si } i = j \\ \text{aleatorio} & \text{si } i \neq j \end{cases}$$

Puede ser simétrica o asimétrica.

- Generación de población:

La población inicial consiste en un conjunto de soluciones candidatas generadas de manera aleatoria. Cada solución corresponde a una permutación válida de las ciudades.

$$P = \{X^{(1)}, X^{(2)}, \dots, X^{(k)}\}$$

donde:

- $k$  es el tamaño de la población.
- Cada  $X^{(i)}$  representa una ruta válida.

Cada solución  $X^{(i)}$  se define como una permutación del conjunto de ciudades:

$$X^{(i)} = (x_0, x_1, \dots, x_{n-1})$$

tal que:

- $x_j \in G = \{0, 1, \dots, n-1\}$ .
- No existen elementos repetidos.
- Se incluyen todas las ciudades exactamente una vez.

- Método de generación:

Las soluciones se generan utilizando un muestreo aleatorio sin reemplazo:

$$X^{(i)} = \text{random.sample}(G, n)$$

Este procedimiento garantiza que cada ruta sea válida dentro del espacio de soluciones.

## Algoritmo 2-OPT

### 2-OPT (Ascenso más empinado):

- Mejora garantizada en cada iteración.
- Elimina cruces visualmente evidentes.
- Complejidad  $O(k \cdot n^3)$ , costosa para  $n > 500$ .
- Puede quedar atrapado en óptimos locales.

### Visualización de la Mejora 2-OPT

Ruta inicial:

0 → 1 → 2 → 3 → 4 → 5 → 0

(costo: 845)

Intercambio:

(i = 1, j = 4)

Invertir segmento [2,4]

Nueva ruta:

0 → 1 → 4 → 3 → 2 → 5 → 0

(costo: 512)

Mejora = 845 - 512 = 333 (39.4%)

## Conclusiones para el TSP

El algoritmo 2-OPT con ascenso más empinado mejora rutas aleatorias aproximadamente en un 40 %, con un tiempo de cómputo reducido.

El uso de heurísticas como 2-OPT permite obtener soluciones eficientes para problemas complejos, sacrificando la optimalidad global.

## Problema de la Mochila 0/1

El problema de la mochila 0/1 consiste en seleccionar un subconjunto de objetos, cada uno con un peso y un valor, de manera que el peso total no supere una capacidad dada y se maximice el valor total obtenido.

## Formulación Matemática

Dados:

- $n$  objetos, donde cada objeto  $i$  tiene peso  $w_i > 0$  y valor  $v_i > 0$ .
- Capacidad máxima de la mochila  $W$ .

Definimos variables de decisión binarias:

$$x_i = \begin{cases} 1 & \text{si el objeto } i \text{ es seleccionado} \\ 0 & \text{en caso contrario} \end{cases}$$

El modelo de programación entera es:

$$\text{Maximizar} \quad \sum_{i=1}^n v_i x_i \quad (1)$$

$$\text{sujeto a:} \quad \sum_{i=1}^n w_i x_i \leq W \quad (2)$$

$$x_i \in \{0, 1\}, \quad \forall i = 1, \dots, n \quad (3)$$

## Algoritmos Implementados

### 1. Greedy

Ordena los objetos por relación valor/peso:

$$\frac{v_i}{w_i}$$

en orden descendente, seleccionando aquellos que quepan en la mochila.

### 2. Backtracking con poda (Branch and Bound)

Explora el árbol de decisión de forma recursiva, utilizando una cota superior fraccionaria para podar ramas que no pueden mejorar la mejor solución encontrada.

### 3. Recocido Simulado

Metaheurística inspirada en el enfriamiento de metales, que acepta soluciones peores con probabilidad:

$$e^{\Delta/T}$$

para evitar óptimos locales.

## Resultados para $n = 20$ objetos

Cuadro 1: Comparativa de algoritmos para la mochila ( $n = 20$ )

| Algoritmo             | Valor obtenido | Tiempo (ms) | Desviación del óptimo |
|-----------------------|----------------|-------------|-----------------------|
| Backtracking (óptimo) | 524            | 2450        | 0 %                   |
| Recocido Simulado     | 518            | 45          | 1.15 %                |
| Greedy                | 487            | <1          | 7.06 %                |

## Conclusiones

1. Para instancias pequeñas ( $n \leq 20$ ), el backtracking con poda es recomendable si se requiere optimalidad absoluta.
2. Para instancias grandes ( $n > 25$ ), el recocido simulado ofrece el mejor compromiso entre calidad de solución y tiempo computacional, superando al algoritmo greedy con una diferencia de calidad aceptable ( $\leq 2$  % del óptimo).
3. El algoritmo greedy, aunque rápido, presenta una desviación del óptimo cercana al 7 % en promedio, lo que lo hace adecuado únicamente para aplicaciones donde la velocidad es crítica y la calidad no es prioritaria.

## Referencias

- [1] Kellerer, H., Pferschy, U., & Pisinger, D. (2004). *Knapsack Problems*. Springer.
- [2] Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G., & Shmoys, D. B. (1985). *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley.
- [3] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671–680.
- [4] Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791–812.
- [5] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.
- [6] Kreher, D. L., & Stinson, D. R. (1999). *Combinatorial Algorithms: Generation, Enumeration, and Search*. CRC Press.